

Reporte de Laboratorio: Arquitectura Multi-Nivel (N-Tier) y Patrón MVC en .NET

Autor: Jorge Rene Velasquez Ruano

Curso: Introducción a la Programación y Computación 2

Fecha: 17 Junio 2026

Parte 1: Fundamentación Teórica y Análisis Crítico

1. El Tránsito hacia los Sistemas Distribuidos y Multi-Capa

- **Limitación del Monolito Local:** Cuando todos los componentes (interfaz, lógica de negocio y almacenamiento) residen en una misma máquina, se presentan problemas graves de escalabilidad (no se pueden aumentar recursos de forma independiente), sincronización (los datos no están centralizados para múltiples usuarios) y mantenimiento (un cambio en una capa afecta a todo el sistema). Además, la disponibilidad depende de un solo punto de fallo.
- **Distinción Layers vs. Tiers:**
 - **Capas Lógicas (Layers):** Son agrupaciones conceptuales de responsabilidades dentro del código (ej. presentación, negocio, datos). Se comunican de forma interna y no implican una separación física.
 - **Niveles Físicos (Tiers):** Representan la ubicación física de los componentes en diferentes servidores o máquinas. Cada nivel puede ejecutar una o más capas lógicas, pero en una arquitectura de 3 niveles cada capa se despliega en un servidor distinto.
- **Responsabilidades en la Arquitectura de 3 Niveles:**
 - **Nivel 1 – Presentación (Presentation Tier):** Maneja la interacción con el usuario (interfaz gráfica, formularios, visualización de datos). Tecnologías comunes: HTML, CSS, JavaScript, frameworks front-end (React, Angular), o vistas Razor en [ASP.NET](#).
 - **Nivel 2 – Aplicación o Negocio (Application Tier):** Contiene la lógica de negocio, validaciones, reglas de procesamiento y orquestación de datos. Se comunica con la capa de presentación y con la de datos. Tecnologías comunes: C#, Java, Python, Node.js con frameworks como [ASP.NET Core](#), Spring Boot, Django.
 - **Nivel 3 – Datos (Data Tier):** Almacena y gestiona la información persistente. Incluye bases de datos (SQL, NoSQL) y sistemas de archivos. Tecnologías comunes: SQL Server, MySQL, PostgreSQL, MongoDB.
- **Seguridad Perimetral:** Exponer directamente el puerto de una base de datos (ej. 3306 para MySQL, 1433 para SQL Server) a Internet es un error crítico porque permite ataques de fuerza bruta, inyección SQL, y acceso no autorizado a datos

sensibles. La buena práctica es colocar la base de datos en una red privada (VPC) y permitir el acceso solo desde el servidor de aplicación a través de una IP interna o mediante un proxy seguro (como un API Gateway). Además, se debe usar autenticación fuerte, cifrado en tránsito (TLS) y mantener actualizados los parches de seguridad.

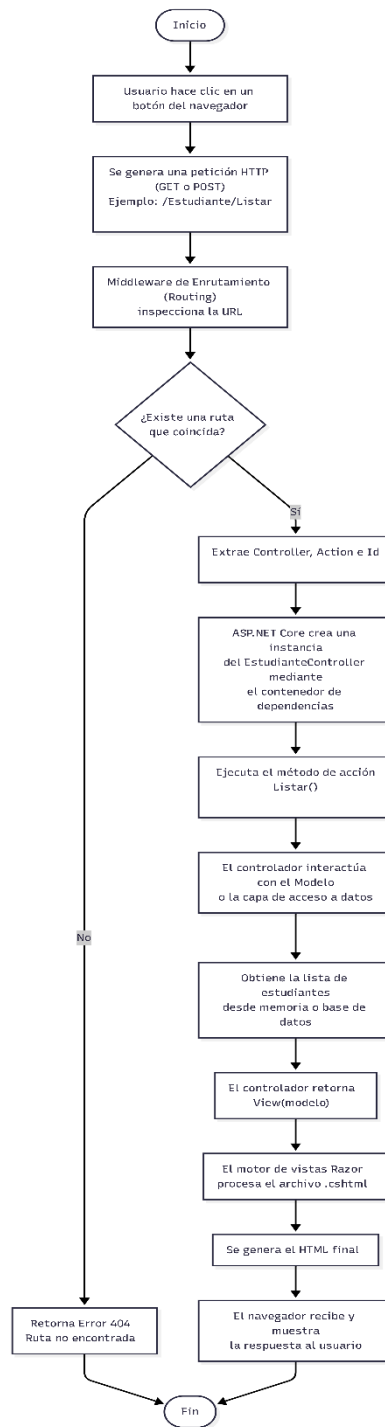
2. Desacoplamiento Lógico con el Patrón MVC

- **Crisis del Código Espagueti:** Mezclar sentencias SQL, lógica matemática y etiquetas HTML en un mismo archivo provoca que el código sea difícil de entender, mantener y probar. Un cambio en la interfaz puede afectar la lógica de negocio, y las pruebas unitarias se vuelven casi imposibles porque cada componente depende de los otros.
- **Separación de Preocupaciones (SoC):**
 - **Modelo:** Representa los datos y la lógica de negocio. No conoce cómo se muestran los datos, es independiente de la interfaz. Se encarga de validaciones, reglas de negocio y acceso a datos (si es necesario). Es reutilizable en diferentes contextos.
 - **Vista:** Es la representación visual de los datos. Se define como una entidad pasiva porque solo muestra información; no contiene lógica de negocio. El tipo de código que tiene estrictamente prohibido contener es: consultas SQL, cálculos complejos, acceso directo a bases de datos o manipulación de estado de la aplicación. Solo debe tener etiquetas HTML y expresiones simples de visualización.
 - **Controlador:** Actúa como intermediario táctico y director de orquesta. Recibe las peticiones del usuario, interpreta los parámetros, invoca al modelo para obtener o modificar datos, y selecciona la vista adecuada para devolver la respuesta. No debe contener lógica de negocio, solo orquestación.
- **Métricas de Ingeniería de Software:**
 - **Alta Cohesión:** Cada componente tiene una responsabilidad única y bien definida. El modelo se enfoca en datos, la vista en presentación y el controlador en el flujo de peticiones.
 - **Bajo Acoplamiento:** Los componentes se comunican a través de interfaces claras (ej. el controlador llama a métodos del modelo, pero no conoce su implementación interna). Esto permite modificar una capa sin afectar a las demás, facilitando pruebas unitarias y mantenimiento.

1. Mapeo Analítico de URLs

URL Entrante del Cliente	Clase Controladora Buscada por el Framework	Método (Acción) Ejecutado	Parámetro id Inyectado
https://ingenieria.usac.edu.gt/ControlAcademico/Login	ControlAcademicoController	Login	(ninguno)
https://ingenieria.usac.edu.gt/Estudiante/Historial/20260123	EstudianteController	Historial	20260123
https://ingenieria.usac.edu.gt/Asignacion/Detalle/10	AsignacionController	Detalle	10
https://ingenieria.usac.edu.gt/Home	HomeController	Index (por defecto)	(ninguno / opcional)

2. Diagramación del Flujo Interactivo



Parte 3: Implementación Práctica - Sistema de Control Académico

El registrar estudiante usa método post para registrarlo

The screenshot shows a web browser window with the address bar displaying 'localhost:7159/Estudiante/Listar'. The page title is 'Actividad3_202405106'. The main content area has a header 'Listado de Estudiantes' and a table with three columns: 'Carné', 'Nombre', and 'Promedio'. The table contains three rows of student data. Below the table is a link 'Actualizar lista'. Further down is a section titled 'Registrar nuevo estudiante' which includes a green success message, three input fields for 'Carné:', 'Nombre:', and 'Promedio:', and a green 'Registrar' button.

Carné	Nombre	Promedio
2026012	Fernando Velasquez	91.5
2026045	Maria Mercedes	84
202405106	Jorge	90

[Actualizar lista](#)

Registrar nuevo estudiante

Estudiante Jorge registrado correctamente.

Carné:

Nombre:

Promedio:

Parte 5: Referencias Bibliográficas

- **Facultad de Ingeniería, USAC. (2026). Sesión 11: Modelado Base y Arquitecturas de Despliegue. Evolución de Sistemas Distribuidos, Fundamentos del Modelo Cliente-Servidor y Diseño Físico Multi-Capas (N-Tier). Laboratorio del curso Introducción a la Programación y Computación 2. Guatemala.**

- Facultad de Ingeniería, USAC. (2026). Sesión 12: Arquitectura y Componentes del Patrón MVC. Desacoplamiento Lógico de Software, Ciclo de Vida de las Peticiones y Enrutamiento en Aplicaciones Interactivas Modernas. Laboratorio del curso Introducción a la Programación y Computación 2. Guatemala.
- Microsoft Docs. (2026). [ASP.NET](https://docs.microsoft.com/en-us/aspnet/core/mvc/overview) Core MVC Overview. Disponible en: <https://docs.microsoft.com/en-us/aspnet/core/mvc/overview>
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.